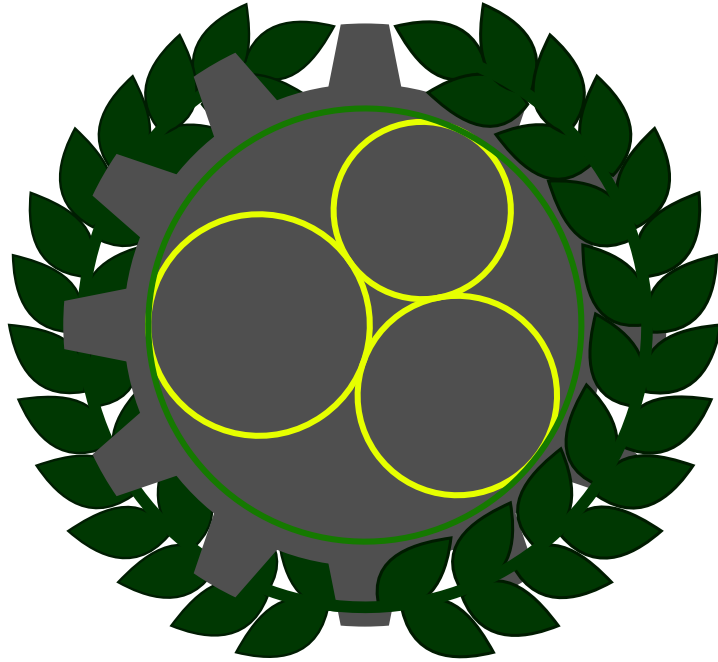




EMPIR Measurement Control User Manual

Version 1.0.0-r0
For EMPIR Measurement Control Version 1.0.0

Alexander Wolfertz, Adrian Losko

2024-08-29

LoskoVision

Contents

1	About this Guide	3
1.1	File Types	3
2	Quick Start Guide	4
2.1	Requirements	4
2.2	Data Acquisition	4
2.2.1	Setup	4
2.2.2	Run Acquisition with Online Data Processing	4
2.3	Post Processing	5
3	Setup	6
3.1	Requirements	6
3.1.1	Hardware	6
3.1.2	Software	6
3.2	Installation	7
4	Usage Guide	8
4.1	Pixel2Photon Application	8
4.1.1	Parameters	8
4.1.2	Examples	8
4.2	Photon2Event Application	10
4.2.1	Parameters	10
4.2.2	Examples	11
4.3	Event2Image Application	12
4.3.1	Parameters	12
4.3.2	Examples	13
4.4	Data Acquisition Script	16
4.4.1	Configuration	16
4.4.2	Set Reconstruction Parameters	16
4.4.3	Run Data Acquisition	16
4.4.4	Examples	17
4.5	Post Processing Script	19
4.5.1	Run Post Processing	19
4.5.2	Examples	19

1 About this Guide

This document contains instructions on how to use the EMPIR measurement control software suite. It only contains information on the usage, not on the details of the algorithms and other inner workings.

1.1 File Types

The software uses several types of files to store intermediate information. They are only intended to be used as input for further processing steps, not to be opened by other programs. The file types are:

- .tpx3 files: These store the pixel activations of the Timepix3 image sensor. They are not generated by the EMPIR software but instead by the serval application used for communicating with the detector. This is the most raw information stored at any point during processing.
- .empirphot files: These store information on reconstructed photons.
- .empirevent files: These store reconstructed events. This is the final event mode data being stored after complete processing.

The final output provided to the user is in the form of images or image series and saved as a TIFF image or TIFF image stack.

2 Quick Start Guide

2.1 Requirements

This software is written for a Gen1 Loskovich LumaCam and an up-to-date Ubuntu 22.04 operating system. The following software needs to be pre-installed (details on how to obtain these can be found in chapter 3.1.2):

- libtiff
- python3 (accessible as "python")
- Serval

2.2 Data Acquisition

2.2.1 Setup

- Follow the detector manual to set up the detector and connect it to the computer.
- Navigate to the EMPIR Measurement Control base directory (the directory containing (among other things) the **bin**, **documentation**, **epics** and **python** folders).
- Copy the `./python/settings_installation_example.py` file to `./python/settings_installation.py` and set all variables in the newly created file according to the installation and the used detector (details can be found in section 3.2).
- Set the "measurement_folder_base_path" variable in the `./dataAcq_single.sh` script to where you want all your measurement data to be saved.
- Set the "measurement_folder_tmp_base_path" variable in the `./dataAcq_single.sh` script to a location where intermediate data needed for processing can be stored temporarily.

2.2.2 Run Acquisition with Online Data Processing

- Navigate to the EMPIR Measurement Control base directory.
- Set the parameters for online processing in the `./parameterSettings.json` file (see sections 4.1, 4.2, and 4.3 for details on the individual options).

- Call the `./dataAcq_single.sh` script with the desired measurement time in seconds and the name for the measurement, e.g.

```
$ ./dataAcq_single.sh 60 nameOfMeasurement
```

for a 60 second measurement with the name `nameOfMeasurement`.

- The resulting image is saved under `"measurement_foler_base_bath"/nameOfMeasurement/final/image`.
- The raw data (e.g. for post processing) is saved in the form of `.tpx3` files in the folder `"measurement_foler_base_bath"/nameOfMeasurement/tpx3Files`.

2.3 Post Processing

- Navigate to the EMPIR Measurement Control base directory.
- Set the parameters for processing in the `./parameterSettings.json` file (see sections 4.1, 4.2, and 4.3 for details on the individual options).
- Find the path (either absolute or relative to the EMPIR Measurement Control base directory) of the folder of the measurement to be processed. This is the folder containing the `tpx3Files` folder. For example if the measurement folder is `/XXX`, all `.tpx3` files of that measurement should be stored like `/XXX/tpx3Files/oneOfTheFiles.tpx3`.
- Call the `./dataReduct.sh` script with the measurement folder path as an argument, e.g.:

```
$ ./dataReduct.sh /XXX
```

3 Setup

3.1 Requirements

3.1.1 Hardware

Data acquisition requires a Gen1 Loskovision LumaCam detector.

3.1.2 Software

The applications included in this package are compiled for a fully up-to-date Ubuntu 22.04 operating system. Other operating systems are currently not supported. All installation and setup instructions are therefore also intended for the Ubuntu operating system.

- The library "libtiff" is required to be installed on the system. It can be installed for example via

```
$ sudo apt-get install libtiff-dev
```

- Python 3 has to be installed and "python" is required to refer to "python3". This is only required for data acquisition. It is not a requirement for post-processing. This can be set up e.g. via

```
$ sudo apt-get install python3  
$ sudo apt-get install python-is-python3
```

- Serval is a server application for communicating with the detector. This software is made for serval version 2.1.6, other versions are not supported. The correct serval version should have been provided together with the LoskoVision LumaCam. This is only required for data acquisition. It is not a requirement for post-processing.
- ImageJ is a software to display images such as the ones generated by this package. It allows a lot of useful operations and analysis to be performed with little effort. It (or an application with similar capabilities) is therefore not a hard requirement but a highly recommended addition. More information can be found under <https://imagej.net/software/fiji/>.

3.2 Installation

Save the entire EMPIR Measurement Control base folder (the folder that contains the `README.md` file) at any location. The path of the project folder is referred to as `<EMPIR Base>` from here on.

The following steps are only required for data acquisition and can be skipped if the installation should only be used for post processing.

- Create a copy of `<EMPIR Base>/python/settings_installation_example.py` and save it as `<EMPIR Base>/python/settings_installation.py`.
- Set the `"nWorkers"` variable in the `<EMPIR Base>/python/settings_installation.py` file. This controls how many threads can be used for on-line data processing. A higher number allows the online processing to better keep up with incoming data and allows measurements at higher hit rates without pausing between measurements to let the processing catch up. However, care should be taken to leave enough computing power for the data acquisition to fetch data from the camera as data can be lost if it is not collected in time. A good guideline for most measurements if no other (resource intensive) software is running is to leave at least 4 physical cores for the data acquisition.
- Set the `"pathServalApp"` variable in the `<EMPIR Base>/python/settings_installation.py` file to the path of the `.jar` file of the serval installation.

4 Usage Guide

The core functionality of the EMPIR package is in the applications in the `<EMPIR Base>/bin` folder. These are described in sections 4.1, 4.2, and 4.3. To facilitate the usage of the applications in the most common circumstances, several bash scripts for data acquisition and post processing are provided with the installation. Their usage is described in sections 4.4, and 4.5. These scripts can also be used as a template for new user-defined scripts.

4.1 Pixel2Photon Application

The `<EMPIR Base>/bin/empir_pixel2photon_tpx3spidr` application reads a single .tpx3 file produced by the serval application and containing a list of pixel activations. From this it reconstructs a list of photons coming from the LumaCam scintillator screen and saves it as a .empirphot file.

4.1.1 Parameters

The parameters for the clustering algorithm can be set via command line flags (see table 4.1 entries with type "Processing"). The *dspace* and *dtime* parameters control the distance in space and time between pixel activations in the individual clusters. Lower values lead to an increase in the number of clusters and a decrease of the size of the individual clusters. The *nPxMin* parameter controls the minimum number of pixel activations a cluster has to have to be considered a photon by the algorithm. Clusters with a lower number of pixel activations are considered to be noise and are discarded.

The *paramsFile* can be used to set all other processing parameters via a .json file. The application expects a JSON object named "pixel2photon" at the document top level. This can contain any combination of the processing parameters as key-value pairs with the key being the same as the corresponding long version of the command line flag without the leading "--".

If the TDC1 input of the Timepix3 camera is used, e.g. as a reference in time of flight applications, *TDC1* should be set (see table 4.1). Without this flag, the signal on the TDC1 input is discarded and cannot be used in further processing.

4.1.2 Examples

Use Parameter Flags

Assuming the input file to be saved as `/path/to/data/file.tpx3`, call:

Type	Flag	Short	Parameter	Description
General	<i>--help</i>	<i>-h</i>		Display the help text
	<i>--license</i>			Display summary license information
I/O	<i>--inputFile</i>	<i>-i</i>	input file path	Set .tpx3 input file
	<i>--outputFile</i>	<i>-o</i>	output file path	Set path to the .empirphot file that should be generated
Processing	<i>--dspace</i>	<i>-s</i>	spatial clustering	Set spatial clustering parameter
	<i>--dtime</i>	<i>-t</i>	temporal clustering	Set temporal clustering parameter
	<i>--nPxMin</i>	<i>-k</i>	min number of pixels	Set minimum number of pixel activations for a photon
	<i>--paramsFile</i>	<i>-f</i>	parameter file path	Set .json file to read processing parameters from
Metadata	<i>--TDC1</i>	<i>-T</i>		Read external trigger inputs on TDC1

Table 4.1: Overview of the command line flags for the Pixel2Photon application

```
$ ./bin/empir_pixel2photon_tpx3spidr -i /path/to/data/file.tpx3
-o /path/to/output/file.empirphot -s 10 -t 1e-6 -k 2
```

The output is saved as `/path/to/output/file.empirphot`. No external trigger input is recorded (e.g. for a measurement at a continuous source).

Use Parameter Settings File

The parameters are saved in a JSON file at `./parameterSettings.json`:

```
{
  "pixel2photon":{
    "dSpace" : 5,
    "dTime" : 500e-9,

    "nPXMin" : 3,

    "TDC1" : true
  }
}
```

Assuming the input file to be saved as `/path/to/data/file.tpx3`, call:

```
$ ./bin/empir_pixel2photon_tpx3spidr -i /path/to/data/file.tpx3
-o /path/to/output/file.empirphot -f ./parameterSettings.json
```

The output is saved as `/path/to/output/file.empirphot`. The external triggers at the TDC1 input are recorded with the data.

4.2 Photon2Event Application

The `<EMPIR Base>/bin/empir_photon2event` application reads a single `.empirphot` file produced by the Pixel2Photon application. From this it reconstructs a list of events and saves it as a `.empirevent` file. Apart from the desired events from particle interactions in the LumaCam scintillator screen, this also contains noise events, e.g. dark noise, stray light, afterglow, and noise from interactions of other types of particles. Filtering out this noise happens later in the processing chain.

4.2.1 Parameters

The parameters for the clustering algorithm can be set via command line flags (see table 4.2 entries with type "Processing"). The *dspace* and *dtime* parameters control the distance in space and time between photons in the individual clusters. Lower values lead to an increase in the number of clusters and a decrease of the size of the individual clusters. The *durationMax* parameter controls the maximum duration (in seconds) any cluster can have.

Type	Flag	Short	Parameter	Description
General	<i>--help</i>	<i>-h</i>		Display the help text
	<i>--license</i>			Display summary license information
I/O	<i>--inputFile</i>	<i>-i</i>	input file path	Set .empirphot input file
	<i>--outputFile</i>	<i>-o</i>	output file path	Set path to the .empirevent file that should be generated
Processing	<i>--dspace</i>	<i>-s</i>	spatial clustering	Set spatial clustering parameter
	<i>--dtime</i>	<i>-t</i>	temporal clustering	Set temporal clustering parameter
	<i>--durationMax</i>	<i>-D</i>	maximum duration [s]	Set maximum duration of individual clusters
	<i>--paramsFile</i>	<i>-f</i>	parameter file path	Set .json file to read processing parameters from

Table 4.2: Overview of the command line flags for the Photon2Event application

The *paramsFile* can be used to set all other processing parameters via a .json file. The application expects a JSON object named "photon2event" at the document top level. This can contain any combination of the processing parameters as key-value pairs with the key being the same as the corresponding long version of the command line flag without the leading "--".

4.2.2 Examples

Use Parameter Flags

Assuming the input file to be saved as /path/to/data/file.empirphot, call:

```
$ ./bin/empir_photon2event -i /path/to/data/file.empirphot
-o /path/to/output/file.empirevent -s 20 -t 5e-6 -D 25e-6
```

The output is saved as /path/to/output/file.empirevent.

Use Parameter Settings File

The parameters are saved in a JSON file at ./parameterSettings.json:

```
{
```

```

    "photon2event":{
        "dSpace_px" : 4,
        "dTime_s" : 100e-6,

        "durationMax_s" : 500e-6
    }
}

```

Assuming the input file to be saved as `/path/to/data/file.empirphot`, call:

```

$ ./bin/empir_photon2event -i /path/to/data/file.empirphot
    -o /path/to/output/file.empirevent -f ./parameterSettings.json

```

The output is saved as `/path/to/output/file.empirevent`.

4.3 Event2Image Application

The `<EMPIR Base>/bin/empir_event2image` application reads multiple `.empirevent` files produced by the Photon2Event application. It filters the events with the aim of only selecting the ones belonging to the desired particle interaction in the scintillator screen. The filtered events are then binned into an image or optionally a sequence of images to provide time-dependent information. The output is saved as a TIFF image or TIFF image stack respectively. Each pixel contains a gray value equal to the number of events in the corresponding bin.

4.3.1 Parameters

The parameters for the filter can be set via command line flags (see table 4.3 entries with type "Filter"). The *nPhotons_min* and *nPhotons_max* are the minimum and maximum numbers of photons in an event accepted by the filter. Events with less than *nPhotons_min* or more than *nPhotons_max* number of photons in them will be disregarded when creating the image. A pulse shape discrimination (PSD) value is calculated for each event. It indicates the spread of an event over time. Larger values indicate more spread out events and lower values indicate more concentrated events. When using a scintillator that produces light with a different temporal distribution for different particle interactions, this value can be used to discriminate between these particles. The *psd_min* and *psd_max* are the minimum and maximum PSD values for an event accepted by the filter. Events with a PSD value outside of this range will be disregarded when creating the image. If one of the values is not set, it indicates no upper or lower bound for the number of photons or the PSD value.

The parameters for binning can be set via command line flags (see table 4.3 entries with type "Binning"). The *size_x* and *size_y* values indicate the number of pixels in x and y direction of the generated image(s). If no value for *size_y* is set, it defaults to the same as *size_x*. The *time_extTrigger* flag controls how the time for binning and the *time_res_s* parameter (if set) is interpreted:

- "ignore": The absolute time t_{abs} relative to the start of the measurement is used for binning. A bin at index i contains events with $i * time_res_s \leq t_{abs} < (i + 1) * time_res_s$.
- "reference": The relative time t_{rel} to the last external trigger is used for binning. A bin at index i contains events with $i * time_res_s \leq t_{rel} < (i + 1) * time_res_s$.
- "frameSync": The absolute time t_{abs} relative to the start of the measurement is used for binning but each bin starts at the time of an external trigger. If t_i is the $i + 1$ st external trigger, the bin at index i contains events with $t_i \leq t_{abs} < t_i + time_res_s$.

If no value $time_res_s$, a single TIFF image with time integrated values is created. If $time_res_s$ is set, $time_limit$ has to be set as well and a TIFF image stack with $time_limit$ images is generated as the output.

The *paramsFile* can be used to set all filter and binning parameters via a .json file. The application expects a JSON object named "event2image" at the document top level. This can contain any combination of the filter and binning parameters as key-value pairs with the key being the same as the corresponding long version of the command line flag without the leading "--".

4.3.2 Examples

All examples assume three .empirevent files saved as /data/a.empirevent, /data/b.empirevent and /data/c.empirevent.

Time-Integrated Image

Create a time-integrated 512 × 512 px image using only the /data/a.empirevent and /data/b.empirevent files:

```
$ ./bin/empir_event2image -i /data/a.empirevent,/data/b.empirevent
-o ./image -x 512 -m 3
```

The output is saved as ./image. Only events with at least 3 photons are used.

Time-Resolved Image Sequence

All data in the /data folder is used:

```
$ ./bin/empir_event2image -I /data -o ./image -x 512
-t 10e-3 -T 300 -m 3
```

The output is saved as a TIFF stack of 300 images with a 10 ms resolution. They represent the first 3 s of the measurement. Only events with at least 3 photons are used.

Type	Flag	Short	Parameter	Description
General	<i>--help</i>	<i>-h</i>		Display the help text
I/O	<i>--inputFiles</i>	<i>-i</i>	input file path list	Add a comma separated list of .empirevent files for processing
	<i>--inputFolder</i>	<i>-I</i>	input folder path	Add all .empirevent files in that folder for processing
	<i>--outputFile</i>	<i>-o</i>	output file path	Set path to the TIFF output file
Filter	<i>--nPhotons_min</i>	<i>-m</i>	min photons	Set minimum number of photons in an event
	<i>--nPhotons_max</i>	<i>-M</i>	max photons	Set maximum number of photons in an event
	<i>--psd_min</i>	<i>-p</i>	PSD min	Set minimum PSD parameter value
	<i>--psd_max</i>	<i>-P</i>	PSD max	Set maximum PSD parameter value
Binning	<i>--size_x</i>	<i>-x</i>	image size in x	Set image size in x
	<i>--size_y</i>	<i>-y</i>	image size in y	Set image size in y
	<i>--time_extTrigger</i>	<i>-E</i>	"ignore", "reference", or "frameSync"	Set the external trigger mode
	<i>--time_res_s</i>	<i>-t</i>	resolution in time [s]	Set the time bin width, output format becomes a TIFF image stack
	<i>--time_limit</i>	<i>-T</i>	number of time bins	Set the number of time bins
Misc	<i>--paramsFile</i>	<i>-f</i>	parameter file path	Set .json file to read filter and binning parameters from
	<i>--fileFormat</i>		"tiff_w4" or "tiff_w8"	Set to "tiff_w8" for TIFF files > 4GB
	<i>--parallel</i>		"true" or "false"	Control usage of multiple threads

Table 4.3: Overview of the command line flags for the Event2Image application

Time of Flight Image

The data has been recoded at a time of flight facility with an external trigger at the TDC1 input for each pulse. It requires the data to have been saved with the `--TDC1` flag. All data in the `/data` folder is used:

```
$ ./bin/empir_event2image -I /data -o ./image -x 512 -E reference  
    -t 10e-6 -T 1000 -m 3
```

The output is saved as a TIFF stack of 1000 images with a $10\text{ }\mu\text{s}$ resolution at `./image`. They represent the summed-up time windows of 10 ms after each pulse. Only events with at least 3 photons are used.

Use Pulse Shape Discrimination

Create a time-integrated image using all data in the `/data` folder:

```
$ ./bin/empir_event2image -I /data -o ./image -x 512 -m 3 -p 5e-6
```

The output is saved as `./image`. Only events with at least 3 photons and a PSD parameter value of at least $10 \cdot 5^{-6}$ are used.

Use Parameter Settings File

The parameters are saved in a JSON file at `./parameterSettings.json`:

```
{  
  "event2image": {  
    "size_x" : 2048,  
    "size_y" : 2048,  
  
    "nPhotons_min" : 3,  
  
    "psd_min" : 5e-6,  
  
    "time_extTrigger" : "ignore"  
  }  
}
```

Create a time-integrated high resolution (2048×2048 px) image:

```
$ ./bin/empir_photon2event -I /data -f ./parameterSettings.json
```

The output is saved as `./image`. Only events with at least 3 photons and a PSD parameter value of at least $10 \cdot 5^{-6}$ are used.

4.4 Data Acquisition Script

4.4.1 Configuration

- Follow the detector manual to set up the detector and connect it to the computer.
- Set the *config_pixel_path* variable in the `<EMPIR Base>/python/settings_installation.py` file to a path to the appropriate .bpc file for the detector that is being used.
- Set the *config_dacs_path* variable in the `<EMPIR Base>/python/settings_installation.py` file to a path to the appropriate .dacs file for the detector that is being used.
- Set the *measurement_folder_base_path* variables in the `<EMPIR Base>/dataAcq_single.sh` and `<EMPIR Base>/dataAcq_mulit.sh` scripts to the path where you want all your measurement data to be saved. This path will be referred to as `<measurement_folder_base>` from here on.
- Set the *measurement_folder_tmp_base_path* variables in the `<EMPIR Base>/dataAcq_single.sh` and `<EMPIR Base>/dataAcq_mulit.sh` scripts to a location where intermediate data needed for processing can be stored temporarily.

4.4.2 Set Reconstruction Parameters

Set the parameters for online processing in the `<EMPIR Base>/parameterSettings.json` file (see sections 4.1, 4.2, and 4.3 for details on the individual options).

4.4.3 Run Data Acquisition

Single Measurement

To run a single measurement, navigate into the `<EMPIR Base>` directory and call the `./dataAcq_single.sh` script with the desired measurement time in seconds (here called *measTime*) and the path relative to `<measurement_folder_base>` under which the measurement should be saved (here called `<measurementFolder>`):

```
$ ./dataAcq_single.sh measTime <measurementFolder>
```

The resulting image after data processing is saved under `<measurement_folder_base>/<measurementFolder>/final/image`. The raw data (e.g. for post processing) is saved in the form of .tpx3 files in the folder `<measurement_folder_base>/<measurementFolder>/tpx3Files`.

Multiple Measurements

Multiple measurements in direct succession can be conducted by using the `<EMPIR Base>/dataAcq_mulit.sh` script. Each individual measurement is saved in a folder of the form `<measurement_folder_base>/MeasurementName_N`, where *MeasurementName* can be set by the user and *N* is an integer that is increased with each successive measurement. The `<EMPIR Base>/dataAcq_mulit.sh` script has to be called from the `<EMPIR Base>` directory with the desired measurement time in seconds (here called *measTime*), *MeasurementName*, and the desired number of measurements (here called *nMeasurements*) as arguments:

```
$ ./dataAcq_mulit.sh measTime MeasurementName nMeasurements
```

The resulting images are saved under `<measurement_folder_base>/MeasurementName_N/final/image`. The raw data (e.g. for post processing) is saved in the form of `.tpx3` files in the folder `<measurement_folder_base>/MeasurementName_N/tpx3Files`.

4.4.4 Examples

Configuration

Example `<EMPIR Base>/python/settings_installation.py` file:

```
nWorkers = 40
pathServalApp="~/TPX3CAM/Server/serv-2.1.6.jar"
config_pixel_path = "~/TPX3CAM/Settings/settings.bpc"
config_dacs_path = "~/TPX3CAM/Settings/settings.bpc.dacs"
```

Singe Time-Integrated Measurement

The parameters are saved in the `./parameterSettings.json` JSON file:

```
{
  "pixel2photon":{
    "dSpace" : 10,
    "dTime" : 1e-6,

    "nPxmin" : 2
  },
  "photon2event":{
    "dSpace_px" : 4,
    "dTime_s" : 100e-6,

    "durationMax_s" : 500e-6
  },
  "event2image": {
    "size_x" : 2048,
```

```

        "size_y" : 2048,
        "nPhotons_min" : 3
    }
}

```

Start a 10 min measurement:

```
$ ./dataAcq_single.sh 600 /data/timeIntegratedMeas
```

The raw .tpx3 files are saved in the /data/timeIntegratedMeas/tpx3Files folder. The generated image is saved as /data/timeIntegratedMeas/final/image.

Multiple Time of Flight Measurements

The parameters are saved in the ./parameterSettings.json JSON file:

```

{
  "pixel2photon":{
    "dSpace" : 10,
    "dTime" : 1e-6,

    "nPxmin" : 2,

    "TDC1" : true
  },
  "photon2event":{
    "dSpace_px" : 4,
    "dTime_s" : 100e-6,

    "durationMax_s" : 500e-6
  },
  "event2image": {
    "size_x" : 2048,
    "size_y" : 2048,
    "time_res_s" : 10e-6,
    "time_limit" : 1000,

    "nPhotons_min" : 3,
    "psd_min" : 5e-6,

    "time_extTrigger" : "reference"
  }
}

```

Start 5 measurements of 10 min each:

```
$ ./dataAcq_mult.sh 600 /data/tofMeas 5
```

The results are saved in the folders `/data/tofMeas_00000` to `/data/tofMeas_00004`. E.g. for the first measurement, the `.tpx3` files are saved in the `/data/tofMeas_00000/tpx3Files` folder and the generated image is saved as `/data/tofMeas_00000/final/image`.

4.5 Post Processing Script

4.5.1 Run Post Processing

- Set the reconstruction parameters in the `<EMPIR Base>/parameterSettings.json` file (see sections 4.1, 4.2, and 4.3 for details on the individual options).
- Locate the measurement folder for the data that should be processed. This is the folder that contains the `tpx3Files` folder. This folder will be called `<measurement_folder>` from here on, i.e. the `.tpx3` files for this measurement should be located in the `<measurement_folder>/tpx3Files` folder.
- Navigate into the `<EMPIR Base>` directory.
- Call the `./dataReduct.sh` script with the measurement folder path as an argument:

```
$ ./dataReduct.sh <measurement_folder>
```
- The resulting TIFF image or image stack is saved under `<measurement_folder>/final/image`

4.5.2 Examples

Add PSD in Post Processing

In this example we add pulse shape discrimination to the time-integrated example in section 4.4.4. The `.tpx3` files are saved in the `/data/timeIntegratedMeas/tpx3Files` folder. The modified `./parameterSettings.json` includes an additional `psdmin` parameter in the `event2image` part:

```
{
  "pixel2photon":{
    "dSpace" : 10,
    "dTime" : 1e-6,

    "nPxmin" : 2
  },
  "photon2event":{
    "dSpace_px" : 4,
    "dTime_s" : 100e-6,
```

```
        "durationMax_s" : 500e-6
    },
    "event2image": {
        "size_x" : 2048,
        "size_y" : 2048,

        "nPhotons_min" : 3,
        "psd_min" : 10e-6
    }
}
```

Start post processing:

```
$ ./dataReduct.sh /data/timeIntegratedMeas
```

The newly generated image replaces the previous image at `/data/timeIntegratedMeas/final/image`.